

Core Software Engineering — Full Curriculum

Eight phases, backend-focused, from first principles to system design

Phase 1 — How Computers Actually Work

CPU & memory model · stack vs heap · compilation vs interpretation · how Node/V8 fits in

Phase 2 — Operating Systems

Processes vs threads · concurrency & scheduling · memory management · syscalls · why Node is single-threaded and what that costs you

Phase 3 — Networking, From the Wire Up

TCP/IP · sockets · how HTTP is actually parsed · DNS · TLS handshake

Capstone: build an HTTP server on raw TCP sockets, no framework

Phase 4 — Databases From First Principles

Relational theory & real SQL mastery · transactions & isolation levels · indexing (B-trees) & query plans · how a DB engine works internally

Capstone: build a tiny key-value store with a write-ahead log and index

Phase 5 — Backend Architecture & System Design

API design principles (REST/gRPC) · caching & invalidation · message queues & event-driven systems · load balancing & horizontal scaling · CAP theorem & replication

Capstone: implement Raft (leader election) or build a load balancer

Phase 6 — Security

Auth done properly (sessions vs JWT vs OAuth) · encryption basics · OWASP top 10 · rate limiting

Phase 7 — Engineering Practice

Testing (unit/integration, TDD) · design patterns & SOLID · clean architecture · observability (logs/metrics/tracing) · CI/CD

Phase 8 — Infrastructure

Linux fundamentals · Docker & what containers actually are (namespaces/cgroups) · deployment pipelines

Each phase ends in a capstone project meant for a portfolio/GitHub — built from understanding, not copied from a tutorial. Order can flex to pair with coursework.